Miniproject 1: Image generation with a diffusion model

Jing Wen, Nikolay Chikin

University of Geneva

Jing.Wen@etu.unige.ch
Nikolay.Chikin@etu.unige.ch

December 19, 2023

Overview

1. Context and problem
2. Background on diffusion model
   - How to generate images?
   - Noising process
   - Denoising process
   - Sampling
3. Theory and implementation
   - The datasets
   - CondBatchNorm2d
   - CondResNet
   - CondUNet
4. Result and Discuss
   - The loss Trends
   - The FID score
   - DEMO

Context and problem

- Goal: To build a diffusion model from scratch to generate images.
- We take inspiration from *ElucidatingtheDesignSpaceofDiffusion – BasedGenerativeModels*, *apaper* by Tero Karras, Miika Aittala, Timo Aila, Samuli Laine, presented at NeurIPS 2022, available here.

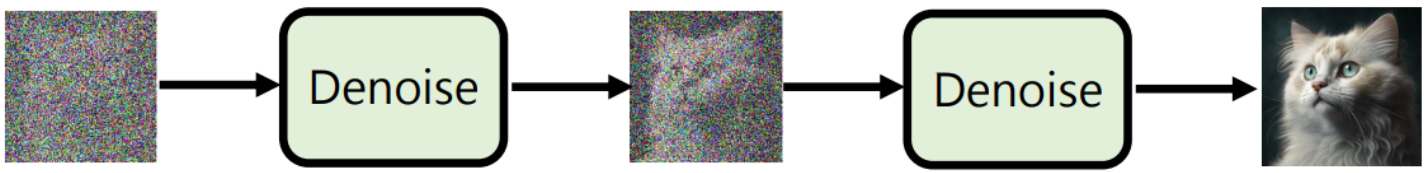
How to generate images?

We have the following two processes:

- Noising process: start from clean images, and add noise until it is indistinguishable from pure noise.



- Denoising process: start from pure noise, iteratively denoiser until it looks like an actual image from the data distribution.



Noising process

We can gradually degrade an image with a sequence of increasing noise levels $\theta_0 = 0 < \theta_1 < \ldots < \theta_T = \theta_{max}$, in a way that the last step is indistinguishable from pure noise.

Algorithm Noising procedure

Sample an image $y \sim p_{data}$ (from the dataset)
Sample a noise level $\sigma \sim p_{noise}$
Sample Gaussian noise $\epsilon \sim N(0, I)$
Obtain degraded sample x by adding noise to the image, $x = y + \sigma\epsilon$

Denoising objective

We are looking for a denoising function D that returns the clean image given the noised image $D(x, \sigma) = y$. Note that D is also taking the noise level σ.

$$\mathcal{L}(\theta) = \mathbb{E}_{y,\sigma,\epsilon}\Big[\|D_\theta(y + \sigma\epsilon, \sigma) - y\|^2\Big] \tag{1}$$

The actual training loss for the network F_θ,

$$\mathcal{L}(\theta) = \mathbb{E}_{y,\epsilon,\sigma}\Big[\| \underbrace{F_\theta\big(c_{\text{in}}(\sigma)\,(y + \sigma\epsilon), c_{noise}(\sigma)\big)}_{\text{Neural network prediction}} - \underbrace{\frac{1}{c_{\text{out}}(\sigma)}\big(y - c_{skip}(\sigma)\,(y + \sigma\epsilon)\big)}_{\text{Neural network training target}} \|^2 \tag{2}$$

Where F_θ is the neural network we want to train.

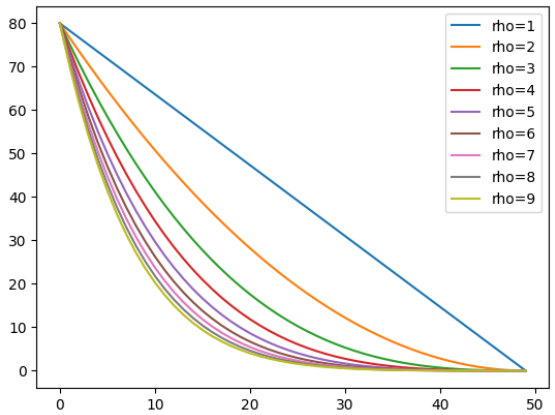
$$D_\theta(x, \sigma) = c_{skip}(\sigma)\,x + c_{out}(\sigma)\,F_\theta(c_{in}(\sigma)\,x, c_{noise}(\sigma)) \tag{3}$$

Sampling

When sampling, we are actually solving the following differential equation, called Probability Flow ODE

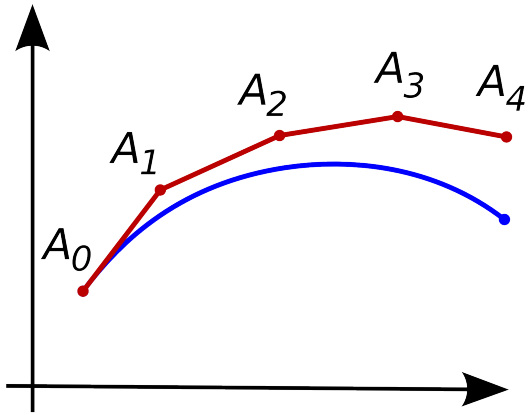
$$dx = -\dot{\sigma}(t)\sigma(t)\nabla_x \log p(x, \sigma(t)dt$$

where $\nabla_x \log p(x, \sigma)$ is also known as the "score" function.




```
def build_sigma_schedule(steps, rho=7, sigma_min=2e-3, sigma_max=80):
    min_inv_rho = sigma_min ** (1 / rho)
    max_inv_rho = sigma_max ** (1 / rho)
    sigmas = (max_inv_rho + torch.linspace(0, 1, steps) * (min_inv_rho - max_inv_rho)
    return sigmas
```

Euler's method for sampling

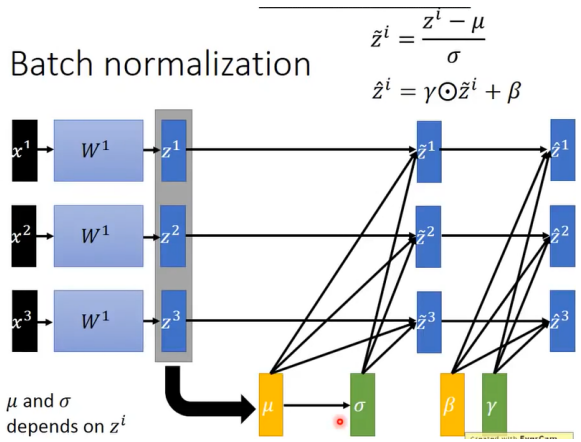


- Principle: Euler's method is a numerical integration technique used for solving ordinary differential equations (ODEs). It approximates the solution by updating the current value based on the slope of the function at that point.
- $dy/dx = f(x, y)$, the iterative step: $y_{n+1} = y_n + h \cdot f(x_n, y_n)$, h is step size.

The datasets

- FashionMNIST:
  methods:
  Padding: Add extra edge pixels around the image so that can increase image size
  Data normalization: Image normalization, mean = 0.5,std=0.5
  We use FashionMNIST for resNet, condResNet, zeroResNet and unet, condUNet, zeroUNet models.
- CelebA
  methods:CenterCrop: Crop at the center of the image, Keep key information
  Resize: Resize the image, square, side length 178
  Data normalization: Image normalization, mean = 0.5,std=0.5
  We use CelebA for the CondUNet model.

CondBatchNorm2d



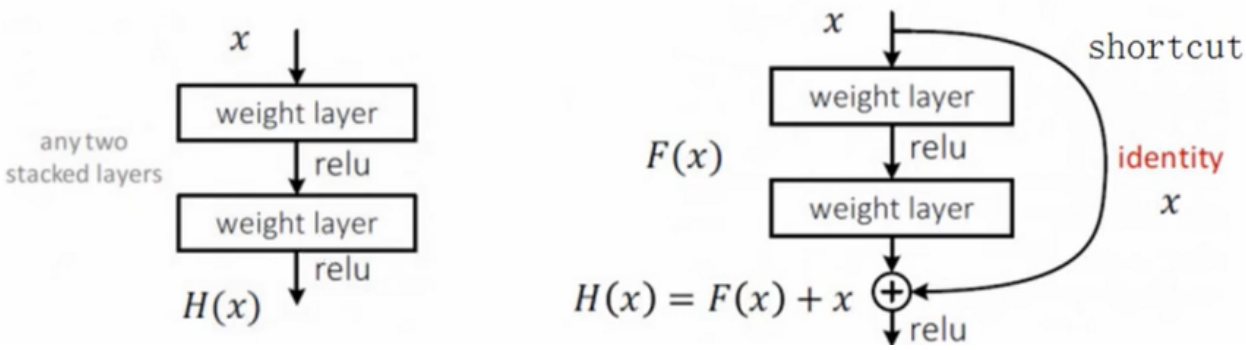
- Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. Input:(N, C, H, W); Output:(N, C, H, W)
- Math:

$$\hat{z}^i = \gamma \odot \widetilde{z}^i + \beta$$

$$\widetilde{z}^i = \frac{z^i - \mu}{\sigma}$$

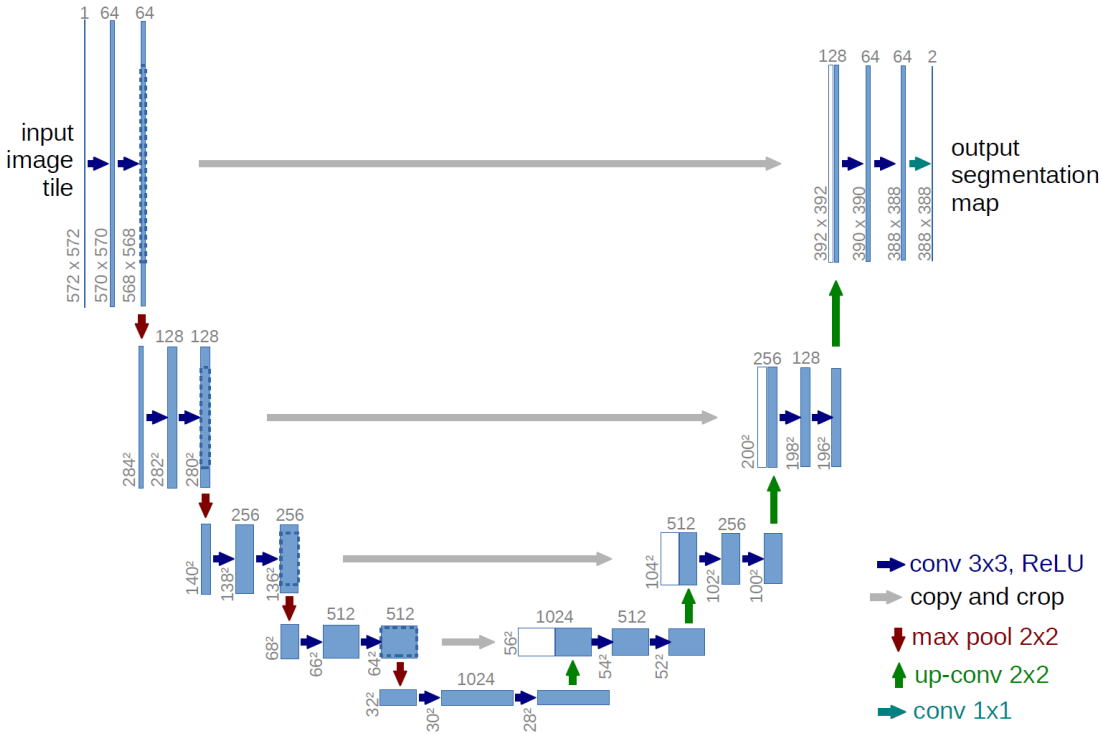
- CondBatchNorm2d: CondBatchNorm2d is a conditional batch normalization layer that can adapt its normalization parameters based on an additional conditional input.
- affine=False: A standard batch normalization layer (nn.BatchNorm2d)

CondResNet



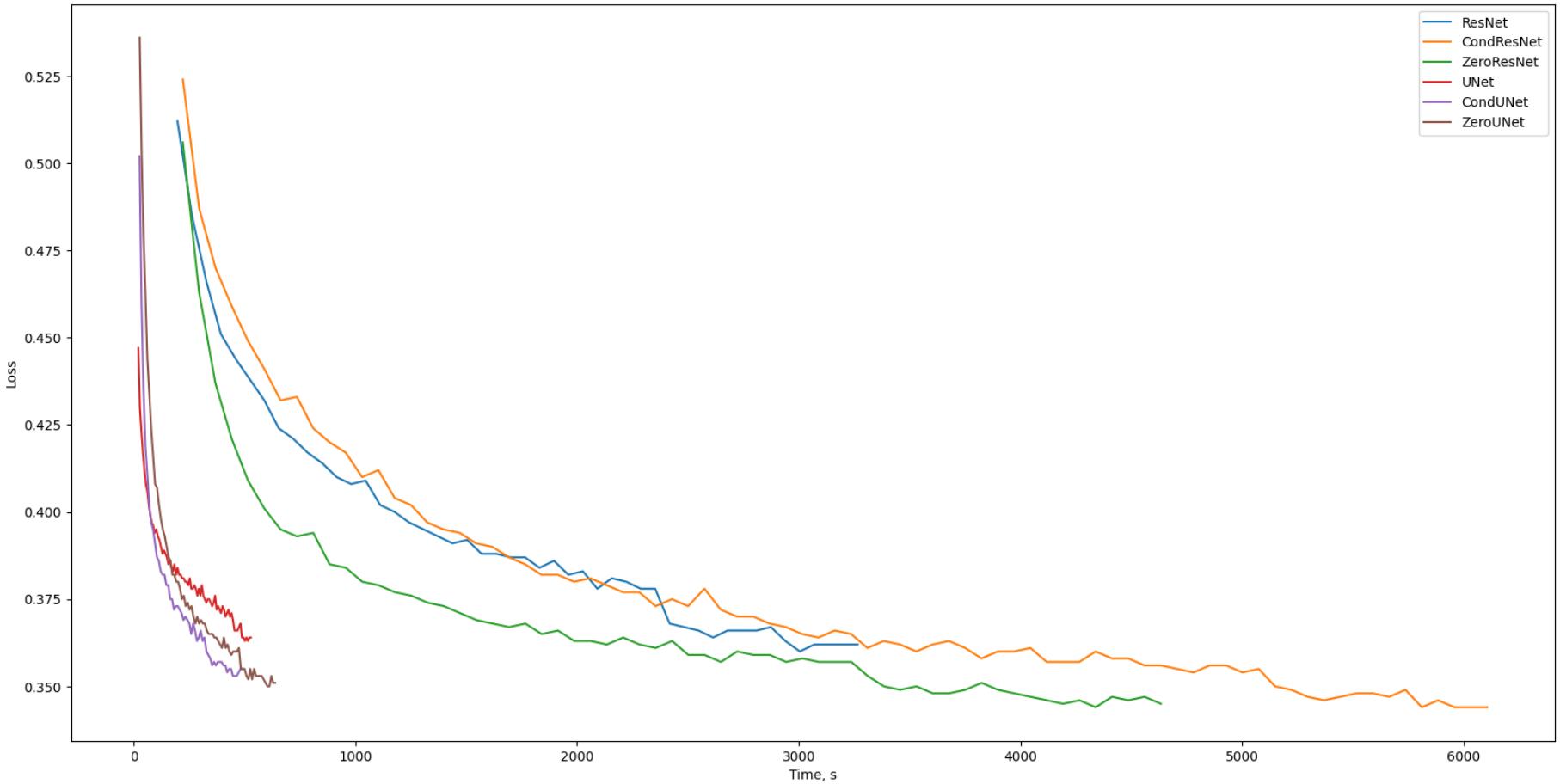
- Residual formula: $x_{l+1)} = x_l + F(x_l, W(l))$
  Through recursion, the expression of the L feature of any deep unit can be obtained: $x_L = x_l + \sum_{l=0}^{L-1} F(x_l, W(l))$
- $output = input + non - lineartransformation(input)$: Correction of the input is achieved by adding the original input to the nonlinearly transformed output. This helps train deep networks and alleviate the vanishing gradient problem;
- $output = input + non - lineartransformation(input, condition)$ Conditional residual connection introduces conditional information based on residual connection

CondUNet

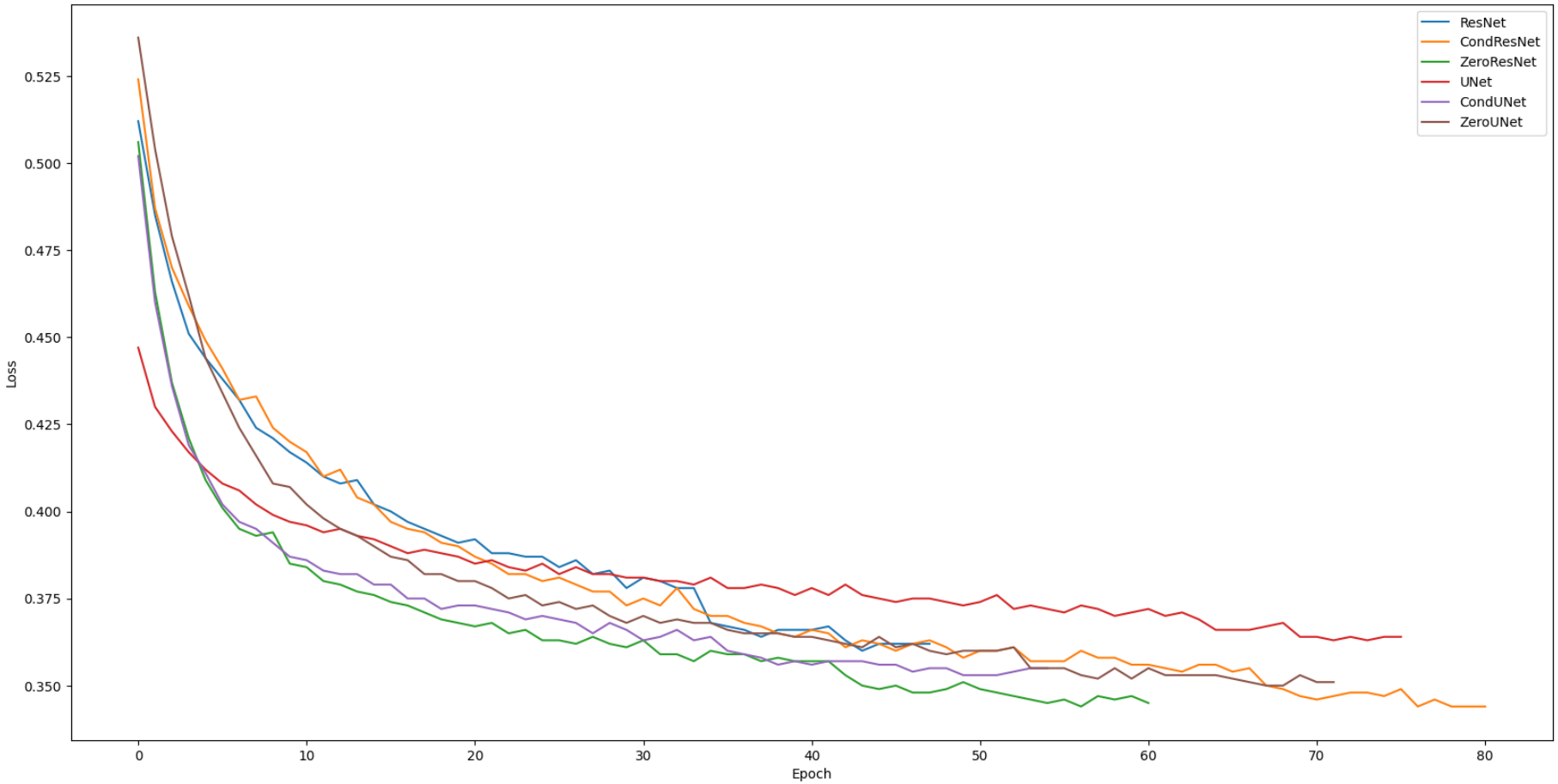


- UNet: Encoder-pooling-decoder architecture to learn feature representations of input images.
- CondUNet: Conditional channels are introduced to generate images based on given conditions.
  Conditional information is integrated at each stage of the encoder and decoder.
- if $cond_channels > 0$, Embed noise information into the model

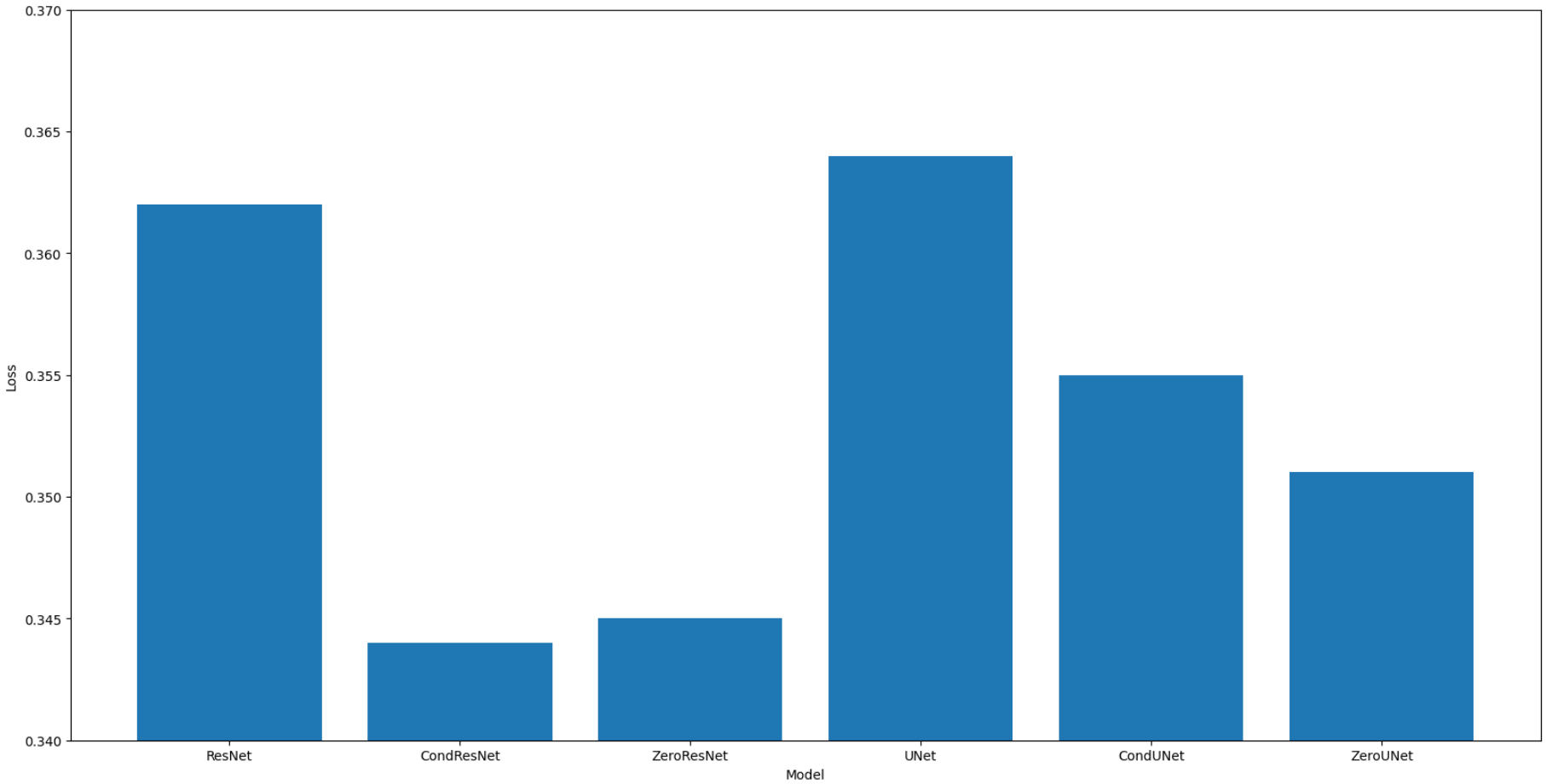
The loss Trends



The loss Trends



The loss Trends



The FID score

Model	Dataset	Epochs	Time, sec	Loss	FID
ResNet	FashionMNIST	50	3266	0.362	32.97
CondResNet	FashionMNIST	83	6106	0.344	30.75
ZeroResNet	FashionMNIST	63	4634	0.345	29.86
UNet	FashionMNIST	78	528	0.364	30.37
CondUNet	FashionMNIST	57	490	0.355	30.77
ZeroUNet	FashionMNIST	74	637	0.351	31.02
CelebA	CelebA	50	17397	0.260	35.85

DEMO
Gitlab here

The End